Financial Dataset Storage and Access Framework

1. Purpose

The framework provides a standard way for financial applications and analytical engines to store, organize, discover, validate, and retrieve structured datasets in Parquet format using object storage.

A processing job may generate:

- intermediate calculation datasets;
- normalized business data;
- derived measures;
- scenario outputs;
- final analytical results;
- reporting datasets;
- warehouse-ready datasets.

The framework manages the physical storage structure, metadata, schema registration, versioning, lifecycle, and access mechanisms.

Application developers should not need to understand bucket paths, partition naming, manifest formats, catalog tables, retention policies, or object-storage write behavior.

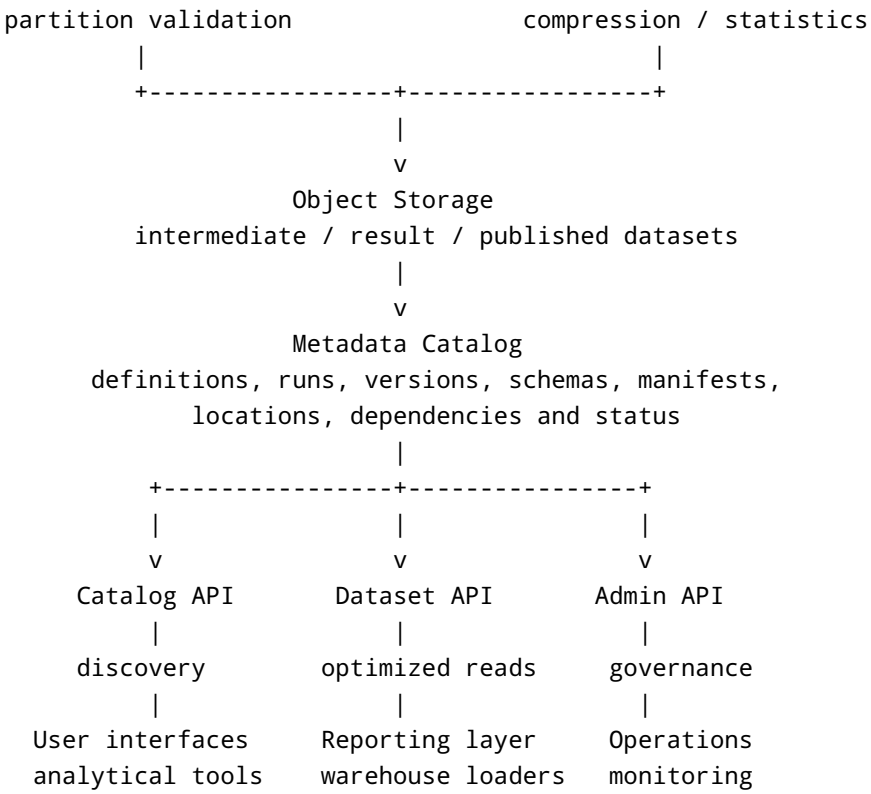
They should provide a dataframe and business context to a client library.

> A processing engine produces named datasets. The framework stores them consistently, records what was created, and makes the data easy to discover and consume.

---

2. High-Level Architecture

```
                  Financial Processing Job
            Risk / Pricing / Reporting / Analytics
                           |
                           |
                  Dataset SDK / Client
             write_dataset(dataframe, context)
                           |
         +-----------------+-----------------+
         |                                   |
         v                                   v
  Dataset Validation                  Dataset Writer
schema / metadata /                 Parquet generation
```



The framework consists of five main components:

1. **Dataset SDK**
   Used by application and analytics developers to write and retrieve datasets.

2. **Object storage dataset store**
   Stores Parquet files, manifests, validation artifacts, and optional dataset-level metadata.

3. **Metadata catalog**
   Stores searchable information about dataset definitions, processing runs, dataset instances, schemas, files, dependencies, and lifecycle status.

4. **Dataset APIs**
   Used by reporting applications, analytical interfaces, warehouse loaders, and operational systems.

5. **Dataset administration services**
   Manage registration, publication, retention, reconciliation, recovery, governance, and access control.

---

3. Core Domain Model

The framework should be organized around datasets and processing runs, not individual files.

3.1 Processing Job

A configured business or analytical calculation.

Examples include:

- portfolio risk calculation;
- market valuation;
- profit-and-loss processing;
- liquidity calculation;
- pricing and sensitivity processing;
- financial reporting preparation;
- reconciliation processing;
- scenario analysis;
- regulatory reporting calculations.

A job may execute repeatedly for different dates, portfolios, accounts, legal entities, scenarios, or reporting periods.

3.2 Processing Run

A single execution of a processing job.

Example:

```
Job:
    daily-portfolio-risk

Run:
    run_id = 01JYB7X8V5P
    business_date = 2026-08-28
    portfolio_id = PF123
    calculation_version = 3.2.1
    status = COMPLETED
```

Every dataset produced during execution belongs to a processing run.

The run provides the common context needed to understand and trace its outputs.

3.3 Dataset Definition

A logical dataset produced or consumed by a processing engine.

Examples:

- normalized_positions;
- instrument_exposures;
- cash_flows;
- market_values;
- pricing_inputs;
- risk_factors;
- scenario_results;
- reconciliation_breaks;
- reporting_summary;
- transaction_details.

A dataset definition describes what the data means. It does not describe one particular physical copy of the data.

3.4 Dataset Instance

A physical materialization of a dataset for a particular processing run and business context.

Example:

```
dataset_name: scenario_results
run_id: 01JYB7X8V5P
business_date: 2026-08-28
portfolio_id: PF123
dataset_version: 1
row_count: 48,250
status: AVAILABLE
```

A dataset instance points to one immutable collection of Parquet files.

3.5 Dataset Class

The initial framework should support three dataset classes.

Intermediate

Datasets generated during processing that may be useful for:

- downstream calculation stages;
- restart and recovery;
- troubleshooting;
- explanation;
- operational diagnostics;

- reconciliation.

Examples:

- normalized inputs;
- enriched transactions;
- grouped positions;
- derived factors;
- staged calculation results.

Result

Completed outputs intended for downstream systems or users.

Examples:

- risk measures;
- profit-and-loss results;
- valuation outputs;
- exception records;
- financial facts;
- scenario results.

Published

A controlled and immutable result approved for official consumption.

Published datasets may be used by:

- external reporting;
- financial statements;
- regulatory reports;
- enterprise warehouses;
- management reporting;
- official business dashboards.

A successful calculation should not automatically make a dataset official.

---

4. Storage Hierarchy

The storage hierarchy should separate business identity from physical optimization.

A recommended structure is:

```
s3://<bucket>/<environment>/
    datasets/
        <domain>/
            <application-or-engine>/
                <dataset-class>/
                    <dataset-name>/
                        schema-version=<version>/
                            business-date=<yyyy-mm-dd>/
                                business-key=<value>/
                                    run-id=<run-id>/
                                        dataset-version=<n>/
                                            part-00000.parquet
                                            part-00001.parquet
                                            manifest.json
                                            _SUCCESS
```

Example:

```
s3://financial-data-prod/datasets/
    risk-management/
        portfolio-risk-engine/
            result/
                scenario-results/
                    schema-version=2/
                        business-date=2026-08-28/
                            portfolio-id=PF123/
                                run-id=01JYB7X8V5P/
                                    dataset-version=1/
                                        part-00000.parquet
                                        part-00001.parquet
                                        manifest.json
                                        _SUCCESS
```

4.1 Benefits of This Structure

The structure provides:

- clear business ownership;
- separation between applications;
- separation between intermediate and official data;
- explicit schema versioning;
- business-date partition pruning;
- isolation between runs;
- support for retries and replay;
- immutable dataset locations;

- manageable retention policies.

4.2 Avoid Excessive Path Partitioning

Not every business attribute should become a directory level.

Avoid creating path partitions for attributes such as:

- currency;
- report type;
- legal entity;
- user;
- scenario name;
- benchmark;
- calculation method;
- transaction status;
- product type;
- processing timestamp.

These should remain catalog attributes or Parquet columns unless they are important and frequently used data-pruning dimensions.

Too many path partitions create:

- small files;
- excessive folders;
- complex lifecycle rules;
- difficult query planning;
- brittle storage conventions.

---

5. Bucket and Environment Model

Use separate buckets for materially different environments.

```
financial-datasets-dev
financial-datasets-test
financial-datasets-prod
```

Within an environment, use controlled logical zones:

```
datasets/intermediate/
datasets/result/
datasets/published/
```

```
system/manifests/
system/quarantine/
system/audit/
system/temporary/
```

A single bucket per environment is usually sufficient for the initial implementation.

Separate buckets may be justified later when different data classes require materially different:

- access policies;
- retention rules;
- regulatory controls;
- encryption keys;
- geographic boundaries.

Recommended controls include:

- public access disabled;
- encryption at rest;
- transport encryption;
- access through managed application roles;
- prefix-based write permissions;
- restricted published-data write access;
- audit logging;
- object versioning where recovery requirements justify it;
- immutable completed dataset versions.

---

6. Dataset Write Lifecycle

The write process must appear transactional to consumers, even though object storage does not provide traditional database transactions.

6.1 Dataset States

```
REGISTERED
    |
    v
WRITING
    |
    v
VALIDATING
    |
    +------> FAILED
    |
```

```
    v
AVAILABLE
    |
    v
PUBLISHED
    |
    v
SUPERSEDED / EXPIRED / DELETED
```

6.2 Safe Write Sequence

1. The client requests a dataset write.
2. The framework generates or validates:
3. run ID;
4. dataset instance ID;
5. dataset version;
6. schema version;
7. target storage location.
8. A catalog record is created with status `WRITING`.
9. Files are written to a run-specific temporary location.
10. The framework validates:
11. schema;
12. required fields;
13. row count;
14. file integrity;
15. business keys;
16. duplicate rules;
17. optional control totals.
18. The framework writes `manifest.json`.
19. The framework writes `_SUCCESS`.
20. The catalog status changes to `AVAILABLE`.
21. Failed or temporary objects are cleaned according to policy.

Consumers must only read datasets whose catalog status is `AVAILABLE` or `PUBLISHED`.

Consumers should not determine completeness by listing files in object storage.

---

7. Manifest Specification

Every dataset instance should have a manifest.

Example:


```
{
  "manifest_version": "1.0",
  "dataset_instance_id": "dsi_01JYB82RZA",
  "dataset_name": "scenario_results",
  "dataset_class": "result",
  "domain": "risk-management",
  "producer": "portfolio-risk-engine",
  "run_id": "run_01JYB7X8V5P",
  "dataset_version": 1,
  "schema_name": "scenario_results",
  "schema_version": 2,
  "business_date": "2026-08-28",
  "portfolio_id": "PF123",
  "calculation_version": "3.2.1",
  "created_at": "2026-08-29T02:16:45Z",
  "status": "AVAILABLE",
  "format": "parquet",
  "compression": "zstd",
  "row_count": 48250,
  "file_count": 4,
  "total_bytes": 18743024,
  "partition_values": {
    "business_date": "2026-08-28",
    "portfolio_id": "PF123"
  },
  "files": [
    {
      "path": "part-00000.parquet",
      "size_bytes": 4681021,
      "row_count": 12000,
      "checksum": "..."
    }
  ],
  "control_totals": {
    "total_market_value": 825000000.12,
    "total_exposure": 797500000.42
  },
  "producer_details": {
    "application": "portfolio-risk-engine",
    "job_name": "daily-risk-calculation",
```

```
    "code_version": "git:83d471a"
  }
}
```

The manifest supports:

- validation;
- recovery;
- reconciliation;
- warehouse ingestion;
- operational diagnosis;
- file integrity checking;
- proving exactly which files form the dataset instance.

The metadata catalog remains the primary discovery mechanism.

The manifest is the physical record stored alongside the files.

---

8. Metadata Catalog

The metadata catalog should initially use a transactional relational database.

Suitable options include PostgreSQL or another approved relational platform.

Object-storage file listings should not be used as the primary catalog.

8.1 Dataset Definition

```
dataset_id
domain
producer
dataset_name
description
dataset_class
owner
business_key_definition
default_retention_policy
current_schema_version
storage_type
status
created_at
updated_at
```

8.2 Dataset Schema

```
schema_id
dataset_id
schema_version
schema_json
compatibility_mode
effective_from
effective_to
created_by
created_at
```

8.3 Processing Run

```
run_id
job_name
producer
business_date
portfolio_id
account_id
legal_entity_id
scenario_id
calculation_version
request_id
started_at
completed_at
status
error_code
error_message
run_metadata_json
```

Not every run will use every business identifier. Optional fields may also be maintained in structured metadata.

8.4 Dataset Instance

```
dataset_instance_id
dataset_id
run_id
dataset_version
schema_version
dataset_class
storage_uri
```

```
manifest_uri
status
row_count
file_count
total_bytes
created_at
available_at
published_at
expires_at
```

8.5 Dataset Partition

```
dataset_instance_id
partition_name
partition_value
```

8.6 Dataset Dependency

```
upstream_dataset_instance_id
downstream_dataset_instance_id
dependency_type
```

Dependency types may include:

```
INPUT
DERIVED_FROM
AGGREGATED_FROM
FILTERED_FROM
RECONCILED_AGAINST
SUPERSEDES
```

8.7 Dataset File

This entity may be optional in the first release if file-level details are stored only in the manifest.

```
dataset_instance_id
file_uri
file_size
row_count
```

```
checksum
min_max_statistics_json
```

8.8 Publication

```
publication_id
dataset_instance_id
publication_status
approved_by
approved_at
publication_reason
supersedes_dataset_instance_id
```

---

9. Schema Management

Parquet stores physical schema information, but that alone is insufficient for financial data governance.

The framework should register and validate schemas explicitly.

9.1 Schema Representation

Store schemas using a framework-neutral JSON model.

Example:

```
{
  "name": "scenario_results",
  "version": 2,
  "fields": [
    {
      "name": "business_date",
      "type": "date",
      "nullable": false,
      "business_role": "dimension"
    },
    {
      "name": "portfolio_id",
      "type": "string",
      "nullable": false,
      "business_role": "identifier"
    },
    {
```

```
      "name": "market_value",
      "type": "decimal(24,8)",
      "nullable": true,
      "business_role": "measure"
    },
    {
      "name": "scenario_change",
      "type": "decimal(24,12)",
      "nullable": true,
      "business_role": "measure"
    }
  ],
  "primary_business_key": [
    "business_date",
    "portfolio_id",
    "instrument_id",
    "scenario_id"
  ]
}
```

9.2 Compatibility Modes

Support three compatibility modes.

Strict

The incoming dataframe must exactly match the registered schema.

Backward Compatible

Changes such as adding nullable fields may be permitted.

None

Breaking changes are allowed only through a new schema version.

Financial result datasets should normally use backward-compatible schema evolution.

9.3 Breaking Changes

Breaking changes include:

- removing a field;
- renaming a field;
- changing field meaning;
- narrowing numeric precision;

- changing business keys;
- changing measurement units;
- changing nullable fields to mandatory;
- changing values from decimal representation to percentage representation;
- changing sign conventions.

9.4 Business Semantics

Schema metadata should contain more than physical data types.

For financial measures, record:

- unit;
- scale;
- currency treatment;
- aggregation behavior;
- sign convention;
- null meaning;
- rounding rule;
- calculation basis.

Example:

```
{
  "name": "scenario_change",
  "type": "decimal(24,12)",
  "unit": "currency_amount",
  "aggregation": "sum",
  "null_semantics": "not_calculated",
  "sign_convention": "positive_is_gain"
}
```

This prevents downstream applications from guessing the meaning of values.

---

10. Client SDK

The first SDK should support Python because many financial calculations and dataframe-processing applications use Python.

Initial dataframe support should include:

- Pandas;
- PyArrow;
- optionally Polars.

10.1 Basic Write API

```
from financial_data import FinancialDataClient
from financial_data.models import DatasetContext

client = FinancialDataClient()

context = DatasetContext(
    producer="portfolio-risk-engine",
    job_name="daily-risk-calculation",
    run_id=run_id,
    business_date="2026-08-28",
    portfolio_id="PF123",
    calculation_version="3.2.1"
)

result = client.write_dataset(
    dataset_name="scenario_results",
    dataset_class="result",
    dataframe=scenario_results_df,
    context=context
)
```

Example response:

```
DatasetWriteResult(
    dataset_instance_id="dsi_01JYB82RZA",
    dataset_version=1,
    schema_version=2,
    storage_uri="s3://...",
    row_count=48250,
    status="AVAILABLE"
)
```

10.2 Required Client Methods

```
start_run(...)
complete_run(...)
fail_run(...)

write_dataset(...)
read_dataset(...)
find_datasets(...)
get_dataset_metadata(...)
```

```
get_schema(...)
validate_dataframe(...)
publish_dataset(...)
```

10.3 Write Options

```
client.write_dataset(
    dataset_name="normalized_positions",
    dataset_class="intermediate",
    dataframe=df,
    context=context,
    options={
        "mode": "create",
        "compression": "zstd",
        "target_file_size_mb": 128,
        "validate_schema": True,
        "capture_column_statistics": True,
        "control_totals": {
            "position_count": len(df),
            "total_market_value": df["market_value"].sum()
        }
    }
)
```

10.4 Idempotency

The client should accept an idempotency key.

```
client.write_dataset(
    dataset_name="scenario_results",
    dataset_class="result",
    dataframe=df,
    context=context,
    idempotency_key="run_01JYB7:scenario_results:v1"
)
```

If a request is retried after a timeout, the framework should return the previously created dataset rather than create a duplicate.

10.5 Developer Responsibilities

The application developer provides:

- dataframe;
- dataset name;
- dataset class;
- processing-run context;
- optional control totals;
- optional business metadata.

The framework provides:

- path construction;
- Parquet serialization;
- compression;
- file sizing;
- schema validation;
- manifest creation;
- catalog registration;
- retries;
- idempotency;
- write completion;
- audit metadata.

---

11. Query and Read APIs

The framework should separate metadata discovery from dataset retrieval.

These are different use cases and should not be combined into one endpoint.

11.1 Catalog Discovery API

Find Datasets

```
GET /v1/datasets
    ?producer=portfolio-risk-engine
    &datasetName=scenario_results
    &businessDate=2026-08-28
    &portfolioId=PF123
    &status=PUBLISHED
```

Example response:

```
{
  "items": [
    {
      "datasetInstanceId": "dsi_01JYB82RZA",
      "datasetName": "scenario_results",
      "runId": "run_01JYB7X8V5P",
      "businessDate": "2026-08-28",
      "portfolioId": "PF123",
      "schemaVersion": 2,
      "datasetVersion": 1,
      "status": "PUBLISHED",
      "rowCount": 48250
    }
  ],
  "nextPageToken": null
}
```


Get Dataset Metadata

```
GET /v1/dataset-instances/{datasetInstanceId}
```

Get Schema

```
GET /v1/datasets/{datasetName}/schemas/{schemaVersion}
```

Get Run

```
GET /v1/runs/{runId}
```

Get Run Outputs

```
GET /v1/runs/{runId}/datasets
```

Get Dependencies

```
GET /v1/dataset-instances/{datasetInstanceId}/dependencies
```

---

12. Efficient Data Retrieval

A normal REST application should not load a large Parquet dataset into memory and convert the entire dataset into JSON.

The framework should support three access patterns.

12.1 Direct Parquet Access

For reporting platforms, warehouse loaders, and trusted analytical clients:

```
POST /v1/dataset-instances/{id}/access
```

Example response:

```
{
  "datasetInstanceId": "dsi_01JYB82RZA",
  "format": "parquet",
  "accessMode": "SIGNED_URLS",
  "files": [
    {
      "url": "...",
      "expiresAt": "2026-08-29T03:00:00Z"
    }
  ]
}
```

Large consumers should read Parquet directly from object storage through:

- temporary credentials;
- assumed roles;
- signed URLs;
- controlled data-access services.

This avoids routing large data volumes through a REST API.

12.2 Query Service

For interactive applications and selective retrieval:

```
POST /v1/query
```

Example request:

```
{
  "datasetInstanceId": "dsi_01JYB82RZA",
  "select": [
    "instrument_id",
    "scenario_id",
    "market_value",
    "scenario_change"
  ],
  "filters": [
    {
      "field": "scenario_id",
      "operator": "=",
      "value": "RATE_UP_100BP"
    }
  ],
  "orderBy": [
    {
      "field": "scenario_change",
      "direction": "DESC"
    }
  ],
  "limit": 500
}
```

The query service should use a columnar query engine.

Possible implementations include:

- DuckDB;
- Trino;
- Amazon Athena;
- another approved analytical query platform.

An embedded DuckDB-based service may be appropriate for an initial implementation when queries are bounded and concurrency is controlled.

12.3 Curated Application APIs

Frequently used business screens should use stable business-oriented endpoints.

Examples:

```
GET /v1/runs/{runId}/summary
GET /v1/runs/{runId}/results?groupBy=product
GET /v1/runs/{runId}/exceptions
GET /v1/runs/{runId}/drilldown?parent=asset-class
```

User interfaces should not need to understand:

- raw storage paths;
- Parquet file names;
- dataset versions;
- schema evolution;
- storage implementation details.

---

13. Query Service Guardrails

A general query API can create performance, security, and cost problems.

The framework should enforce:

- queries only against registered datasets;
- schema-based column validation;
- structured filters rather than arbitrary SQL;
- maximum row limits;
- maximum bytes scanned;
- execution time limits;
- mandatory pagination;
- role-based column access;
- concurrency quotas;
- query cancellation;
- complete query auditing;
- result caching;
- predicate pushdown;
- column projection;
- partition pruning.

Unrestricted SQL should not be exposed through a general external API.

---

14. Parquet Generation Standards

14.1 File Size

The framework should avoid both extremely large files and large numbers of tiny files.

An initial target is:

```
64 MB to 256 MB compressed per Parquet file
```

Small datasets may be written as one file.

14.2 Compression

Use a standard analytical compression algorithm such as:

- Zstandard;
- Snappy.

The framework should have one default compression standard.

Individual applications should not select different compression formats without a justified reason.

14.3 Row Groups

Row-group sizing should be managed centrally to support:

- predicate pushdown;
- column projection;
- efficient scanning;
- reasonable memory use.

14.4 Sort Order

Datasets used frequently for filtered reads may benefit from consistent sorting before write.

Examples:

```
business_date
portfolio_id
account_id
instrument_id
scenario_id
```

Sort order should be part of dataset configuration rather than an individual developer decision.

14.5 Statistics

Capture selected statistics such as:

- row count;
- file count;
- total bytes;
- null counts;
- minimum and maximum values;
- distinct counts for selected low-cardinality dimensions;
- control totals;
- business-key duplicate counts.

Avoid calculating expensive statistics for every column by default.

---

15. Versioning Model

The framework must distinguish between several kinds of version.

```
Calculation version
    Version of the business logic or processing engine.

Schema version
    Version of the dataset structure and business meaning.

Dataset version
    Revised physical materialization of the output.

Publication version
    Official version selected for downstream consumption.
```

Example:

```
calculation_version = 3.2.1
schema_version = 2
dataset_version = 3
publication_version = 1
```

A rerun should normally create a new run ID.

A controlled correction within the same run may create a new dataset version.

Prior versions should remain traceable.

Published data should not be overwritten.

---

16. Publication Model

Completed data should not automatically become official data.

Recommended lifecycle:

```
AVAILABLE
    |
    | validation and approval
    v
PUBLISHED
    |
    | corrected or replaced
    v
SUPERSEDED
```

Publication should record:

- publisher identity;
- publication timestamp;
- validation status;
- prior version superseded;
- reason for publication;
- downstream distribution status.

Official reporting and warehouse processes should normally consume `PUBLISHED` datasets.

Preliminary analytical users may consume `AVAILABLE` datasets when authorized.

---

17. Intermediate Data Retention

Intermediate data can grow quickly.

Each intermediate dataset should use one of three retention classes.

Ephemeral

Retained for several days.

Used for temporary processing and restart support.

Diagnostic

Retained for several weeks or months.

Used for investigation, comparison, and operational support.

Reproducible

Retained alongside official results.

Used when the dataset is required for:

- audit;
- explanation;
- replay;
- regulatory evidence;
- model validation.

Example:

```
{
  "retentionClass": "DIAGNOSTIC",
  "retentionDays": 30,
  "replayRequired": true
}
```

Dataset definitions should identify whether the data is:

- needed for restart;
- needed for explanation;
- needed for audit;
- reproducible from upstream sources;
- useful only for troubleshooting.

Not every intermediate dataframe should be retained indefinitely.

---

18. Failure and Recovery

The framework should explicitly handle common failure scenarios.

Partial File Write

The dataset remains `WRITING` or becomes `FAILED`.

Consumers cannot discover it as available.

Metadata Registration Succeeds but File Write Fails

The catalog entry is marked `FAILED`.

Cleanup processes remove temporary or orphaned files.

Files Are Written but Catalog Completion Fails

A reconciliation process uses manifests to identify and recover incomplete registrations.

Duplicate Retry

The idempotency key returns the existing dataset instance.

Schema Mismatch

The write fails before the dataset becomes available.

Invalid data is deleted or moved to quarantine according to policy.

Run Fails After Some Intermediate Datasets Succeed

Completed intermediate datasets remain linked to the failed run and follow their defined retention policy.

Consumer Reads During a Write

Consumers can only discover `AVAILABLE` or `PUBLISHED` dataset instances.

---

19. Security Model

Authorization should support multiple control levels.

```
Environment
Domain
Producer
```

```
Dataset class
Dataset name
Portfolio
Account
Legal entity
Column sensitivity
```

Example roles include:

- processing-engine writer;
- analytical reader;
- reporting-application reader;
- warehouse loader;
- dataset publisher;
- support operator;
- catalog administrator.

The SDK should obtain credentials through the runtime environment.

Static object-storage credentials must not appear in application code.

---

20. Operational Observability

The framework should produce operational metrics including:

```
dataset writes started
dataset writes completed
dataset writes failed
write duration
validation duration
bytes written
rows written
file count
small-file count
catalog API latency
query latency
query bytes scanned
query rows returned
publication failures
schema violations
expired datasets pending deletion
orphan files detected
reconciliation failures
```

Every write and query should be traceable using identifiers such as:

```
request_id
run_id
dataset_instance_id
correlation_id
producer
portfolio_id
business_date
```

---

21. Service Specifications

21.1 Dataset Registry Service

Responsibilities:

- register dataset definitions;
- register schemas;
- enforce naming standards;
- manage compatibility rules;
- assign dataset identifiers;
- record ownership;
- record retention policy;
- manage active and retired definitions.

21.2 Run Management Service

Responsibilities:

- start and complete processing runs;
- record processing context;
- associate datasets with runs;
- maintain run status;
- record errors;
- track upstream and downstream dependencies.

21.3 Dataset Write Service

Responsibilities:

- generate storage paths;
- provide write authorization;
- validate schemas;

- control file sizing;
- generate manifests;
- complete dataset registration;
- enforce idempotency;
- reconcile incomplete writes.

The first implementation may place much of this capability inside the Python SDK rather than a separate network service.

21.4 Catalog Service

Responsibilities:

- dataset discovery;
- metadata retrieval;
- schema retrieval;
- run-output lookup;
- latest-version resolution;
- latest-published-version resolution;
- dependency lookup;
- ownership lookup.

21.5 Query Service

Responsibilities:

- selective data retrieval;
- filtering;
- projection;
- pagination;
- output as JSON, Arrow, CSV, or Parquet;
- query resource controls;
- security enforcement;
- caching.

21.6 Publication Service

Responsibilities:

- validate publication eligibility;
- approve or reject publication;
- mark datasets as published;
- manage supersession;
- emit publication events;
- notify warehouse and reporting consumers.

---

22. API Specification

Processing Runs

```
POST   /v1/runs
GET    /v1/runs/{runId}
POST   /v1/runs/{runId}/complete
POST   /v1/runs/{runId}/fail
GET    /v1/runs/{runId}/datasets
```

Dataset Definitions

```
POST   /v1/dataset-definitions
GET    /v1/dataset-definitions
GET    /v1/dataset-definitions/{datasetId}
POST   /v1/dataset-definitions/{datasetId}/schemas
GET    /v1/dataset-definitions/{datasetId}/schemas
```

Dataset Instances

```
POST   /v1/dataset-instances
GET    /v1/dataset-instances/{instanceId}
POST   /v1/dataset-instances/{instanceId}/complete
POST   /v1/dataset-instances/{instanceId}/fail
POST   /v1/dataset-instances/{instanceId}/publish
GET    /v1/dataset-instances/{instanceId}/manifest
POST   /v1/dataset-instances/{instanceId}/access
```

Discovery

```
GET    /v1/datasets
GET    /v1/datasets/latest
GET    /v1/datasets/{datasetName}/schema
```

Queries

```
POST   /v1/queries
GET    /v1/queries/{queryId}
```

```
DELETE /v1/queries/{queryId}
GET    /v1/queries/{queryId}/results
```

---

23. Plain Parquet Versus Table Format

There are two main implementation options.

Option A: Managed Parquet Framework

The framework manages:

- storage paths;
- manifests;
- schemas;
- versions;
- metadata;
- publication;
- lifecycle.

Advantages:

- relatively simple;
- easy integration with Python;
- direct control over storage;
- suitable for run-oriented financial datasets;
- works well with DuckDB and similar query engines;
- lower initial platform complexity.

Disadvantages:

- the framework must implement versioning;
- file compaction requires custom logic;
- updates and deletes require framework support;
- SQL access across many runs requires catalog integration.

Option B: Table Format

A table format such as Apache Iceberg can manage logical datasets as tables with snapshots.

Advantages:

- table-level transaction semantics;
- schema evolution;
- snapshot isolation;

- partition evolution;
- standardized file management;
- easier access from distributed SQL platforms.

Disadvantages:

- additional platform dependencies;
- greater operational complexity;
- temporary run-specific datasets may not need full table semantics;
- developers still require a higher-level run and dataset framework.

Recommendation

Start with a managed Parquet framework.

Do not expose path conventions directly through the client or catalog contract.

Support a storage-type field so selected datasets can later use a table format.

```
storage_type:
    PARQUET_MANIFEST
    ICEBERG_TABLE
```

This allows temporary and run-specific datasets to remain simple while high-volume shared datasets may later use table semantics.

---

24. Recommended Delivery Phases

Release 1: Reliable Dataset Storage

Build:

- dataset registration;
- schema registration;
- run registration;
- Python SDK;
- Parquet writer;
- path generation;
- manifest generation;
- schema validation;
- catalog integration;
- available-state management;
- metadata APIs;
- direct Parquet access;

- basic retention.

Prove the framework with one processing application.

Release 2: Consumption and Publication

Add:

- publication workflow;
- query service;
- interactive application support;
- warehouse-loading integration;
- dependency tracking;
- control-total validation;
- operational dashboards;
- automated recovery.

Release 3: Scale and Governance

Add:

- query caching;
- file compaction;
- fine-grained access controls;
- advanced schema compatibility;
- cross-dataset querying;
- automated lifecycle processing;
- selected table-format datasets;
- cost and usage governance.

---

25. Minimum Developer Experience

A developer should be able to use the framework as follows:

```
run = client.start_run(
    producer="portfolio-risk-engine",
    job_name="daily-risk-calculation",
    business_date=business_date,
    portfolio_id=portfolio_id,
    calculation_version=version
)

client.write_dataset(
    dataset_name="normalized_positions",
```

```
    dataset_class="intermediate",
    dataframe=positions_df,
    context=run.context
)

client.write_dataset(
    dataset_name="scenario_results",
    dataset_class="result",
    dataframe=scenario_results_df,
    context=run.context,
    control_totals={
        "total_market_value": scenario_results_df["market_value"].sum()
    }
)

client.complete_run(run.run_id)
```

The developer should not need to write code such as:

```
df.to_parquet(
    "s3://bucket/prod/results/date=.../portfolio=.../run=..."
)

# Build path conventions
# Write a manifest
# Register schema
# Update catalog
# Handle retry
# Handle partial failure
# Apply retention rules
```

Removing these concerns from processing applications is the primary purpose of the framework.

---

26. Architecture Principles

1. The dataset, not the file, is the primary unit of management.

2. Every dataset instance belongs to a processing run.

3. Logical dataset identity is separate from physical storage location.

4. Completed dataset versions are immutable.

5. Consumers only read catalog-approved dataset states.
6. Dataset schemas include business meaning, not only technical types.
7. Large datasets are accessed directly or through a columnar query engine.
8. Intermediate, result, and published datasets have different trust and retention policies.
9. The SDK hides infrastructure and storage conventions from application developers.
10. Object-storage listings are not a metadata catalog.
11. Publication is a controlled business action, not merely a technical write completion.
12. The initial design should be simple enough to implement, but should not prevent later adoption of table formats or distributed query platforms.

---

27. Next Design Artifacts

The next level of design should include:

1. logical metadata model;
2. detailed Python SDK interface;
3. dataset naming standards;
4. schema-definition standard;
5. storage-path generation rules;
6. dataset-write sequence diagram;
7. dataset-read sequence diagram;
8. failure-recovery sequence diagram;
9. publication workflow;
10. first-release component deployment design.